

Sandboxing

A hands-on course in building agents by building their harness

Lesson 5

It runs every command, as you

Since Lesson 2, every command the model asks for runs:

- on your machine
- as you, with your files, your network and your credentials
- and nothing asks first

The setup warned you to run it somewhere you can afford to lose. This layer makes that true wherever it runs.

Sandboxing decides where commands run

Instead of your machine, the harness runs them in a box:

- its own filesystem, and a limited view of yours
- no way out to the network
- a fixed share of memory and processor, and a time limit
- thrown away when the run is over

It doesn't decide whether a command *should* run. It decides what a command can reach *when* it runs.

Built on output: same place, a different choice

- Output acts on what the model said: the model asks for a command, output runs it (Lesson 2)
- `subprocess.run` was already a choice of where: it means "right here"
- Sandboxing makes a different choice in the same place
- A production layer adds hardening, not a new primitive

The kernel enforces the walls

- A guardrail is code reading a command and deciding
- A container's limits are enforced by the operating system's kernel
- They hold on whatever runs inside, however the command is spelled
- `rm -rf /` written a hundred different ways can still only see the box's filesystem

Docker is the tool quark uses to make the box. It's one of several; the idea is the same in all of them.

Every limit is a setting on the container

- **Lives for the run:** started before the loop, idle (`sleep infinity`); each tool call is a `docker exec`
- **Sees:** only the folder quark started in, at the same path; your environment stays out, API key included
- **Reaches:** nothing (`--network none`)
- **Gets:** 512 MB of memory, a share of processor, 128 processes, 30 seconds a command
- **Changes:** read-only root, a scratch `/tmp` , no capabilities, your user ID

The damage stops at the wall

- It doesn't make the model behave
- A command can still do what's allowed inside, including deleting the mounted folder
- The machine around the box, your other files and your credentials are out of reach
- If the box can't be started, quark stops: there's no fallback to the host

The box: one container for the whole run

Lesson 4's `quark.py` plus 10 lines. `atexit` joins the imports, and the box goes in the output section (abridged: `...` stands for the flags on the last slide):

```
IMAGE, TIMEOUT = "python:3.13-slim", 30
box = f"quark-{os.getpid()}"
def sandbox():
    where = os.getcwd()
    up = subprocess.run(["docker", "run", "-d", "--rm", ..., IMAGE, "sleep", "infinity"], ...)
    if up.returncode: sys.exit(f"[no sandbox, so nothing runs: {up.stdout.strip()}]")
    atexit.register(lambda: subprocess.run(["docker", "rm", "-f", box], ...))
```

`sandbox()` is called once, at the top of `# — control flow —`.

One line moves the command into the box

`subprocess.run(cmd, shell=True)` became this (abridged):

```
done = subprocess.run(["docker", "exec", box, "timeout", "-s", "KILL", str(TIMEOUT),  
                      "sh", "-c", block.input["cmd"]], ...)  
if done.returncode == 137: done.stdout += f"\n(killed: ran over {TIMEOUT} seconds or ...)"
```

- Same shell, same merged output and exit code, but in the box under a 30-second limit
- `-s KILL` : a command that ignores polite requests is still stopped
- Exit 137 is a time-out *or* out of memory; quark can't tell, so it tells the model both

Everything else is Lesson 4's, on purpose

- The prompt still tells the model bash reaches "the whole system"; inside the box, that's true
- The model finds out how big the box is by running into its walls
- Telling it up front would be a decision about context, and this layer doesn't make it
- The episode is still written by the harness, outside the box

Run it: one write works, three hit walls

In a scratch folder: write `hello.txt`, then try `/etc`, `example.com` and the API keys. The model's one long command and its answer are cut (shortened):

```
hi
--- etc
sh: 1: cannot create /etc/testfile: Read-only file systemexit 2
--- net
urllib.error.URLError: <urlopen error [Errno -3] Temporary failure in name resolution>
--- keys
no
--- who
uid=0(root) gid=0(root) groups=0(root)
root
/tmp/demo
/.dockerenv
```

Afterwards: no box, and only what it was allowed to leave

Being root didn't let me write to `/etc`, because the filesystem is read-only. The network is cut off as well. The sandbox limits what I can do, not the user account.

`docker ps -a` lists no containers. The folder holds:

```
hello.txt  
notes.txt
```

Lesson 4's memories still work: `.quark/` lives in the mounted folder, on the host.

A killed command doesn't take the run with it

A copy with `TIMEOUT` set to 5 (shortened):

```
$ sleep 60
```

```
(killed: ran over 5 seconds or out of memory)
```

```
$ python3 -c 'x = bytearray(2*1024**3); x[:] = b"a"*len(x)'
```

```
Killed
```

```
(killed: ran over 5 seconds or out of memory)
```

```
$ ls
```

```
hello.txt
```

```
notes.txt
```

Going further: what else a sandbox can be

- **The box:** a container, gVisor, a Firecracker micro-VM, a full VM, or the OS's own process sandbox
- **What it sees and reaches:** a read-only mount or a copy; an allow list, or a proxy that adds credentials
- **What it gets, and how long it lives:** disk, output, time per run; one box per command, run or session
- **How results get out:** a shared folder, a person says yes, or a patch or pull request
- **Where it runs:** next to the harness, or somewhere else entirely

It can be a product on its own

Hosted sandboxes like **E2B**, **Modal** and **Daytona** start an isolated machine on request, with a way to run commands in it and read files out.

If you're sending your agent's commands to one of those, it's this layer, and the choices on the last slide are theirs.

`sandboxing.py`: the box works on a copy

Lesson 3's agent loop, so the sandbox is all there is to look at:

- `start()` copies the project in, leaving out everything in `KEEP_OUT`
- Stricter `LIMITS`: runs as `nobody`, with small sized scratch disks
- Each command's output is cut at `MAX_OUT` characters
- `review()` lists what changed in the box and asks; only a `y` copies it to `./sandbox-out`

```
KEEP_OUT = [".env", ".git", ".venv", ".quark", "__pycache__"] # never copied in
```

The model deleted the files; the host still has them

The task: write `stats.py`, then delete `data.csv` and `notes.txt`, and look for `.env`. I answered `y` to `review()`. The folder afterwards, then `sandbox-out` (shortened):

```
.env
data.csv
notes.txt
sandbox-out
---
result.txt
stats.py
7.0
```

The rule: run commands where they can't do lasting damage

- Make the box before the loop
- Aim the tool at it in place of `subprocess.run` on the host
- Limit what it can see, reach, use and change with the operating system, not with checks on the text of a command
- Throw it away when the run ends

What sandboxing never does

It sits inside output, changing where a tool runs, and leaves the other primitives alone:

- **Model interface:** still talks to the model from outside the box; the API key never goes in
- **Control flow:** the same loop
- **Input:** still brings back what a command printed, the same way
- **Context:** untouched; the model finds the walls by walking into them

What's missing: deciding whether it runs

- The box limits what a command can reach, not whether it runs
- Inside it everything still runs unasked, including `rm -rf` on the mounted folder: your real project
- Nothing lets you say no to one command, or break in while it works
- Nothing stops a run that loops forever, spending your money one call at a time

Next: Guardrails

Lesson 6